

**International Institute of Information Technology
Hyderabad**

Computer Science and Engineering

EcoTwin

**Ecosystem Analog Search and Forecasting Using
Geospatial Foundation Models**

Project Report

Submitted by

Your Name

Your Roll Number

Under the guidance of

Supervisor Name

Abstract

EcoTwin is an end-to-end geospatial ecosystem analog discovery platform designed around the question: *which ecosystems elsewhere exhibit similar spatial structure, and what can their historical trajectories tell us about a target ecosystem?*

The system combines a static lake inventory with multi-year Sentinel-2 observations, spatial tiling, environmental indicators, geospatial foundation-model embeddings, vector similarity search, and temporal ecosystem profiles. HydroLAKES polygons provide the initial ecosystem boundaries and metadata. For each selected lake, the pipeline projects and buffers the geometry, constructs a manageable imagery grid, retrieves annual Sentinel-2 composites from Google Earth Engine, stores the resulting GeoTIFFs locally, and subsequently subdivides the merged imagery into approximately one-kilometre analysis cells. Cells with insufficient lake coverage are discarded.

The retained image cells are converted into the six-band input required by the Prithvi geospatial foundation model. The resulting 768-dimensional representations are stored in PostgreSQL using pgvector. A query ecosystem can then be compared against the indexed collection using cosine similarity, producing a ranked set of analogous ecosystems. The web application exposes these results through an interactive map and profile views, together with historical NDVI, NDWI, and NBR trajectories.

The project was implemented iteratively over approximately two months. A substantial part of the engineering work involved making large-scale Earth Engine extraction reliable, handling historical Sentinel-2 collection differences, reducing export sizes, merging local raster tiles, controlling concurrency, validating embeddings, and designing a database capable of preserving both spatial geometries and vector features. The forecasting layer is designed as an analog-based, non-parametric extension in which historical trajectories of retrieved twins provide empirical evidence for future ecosystem behaviour rather than requiring synthetic image generation.

Keywords: Geospatial Foundation Models, Prithvi, Remote Sensing, Sentinel-2, HydroLAKES, Ecosystem Analogs, Vector Search, pgvector, NDVI, NDWI, NBR, Temporal Analysis

Contents

Abstract	1
1 Introduction	8
1.1 Background	8
1.2 Problem Statement	8
1.3 Core EcoTwin Hypothesis	9
1.4 Research Objectives	9
1.5 Project Evolution During Development	10
1.6 Contributions	10
1.7 Report Organization	10
2 Data Sources and Data Engineering	11
2.1 Overview	11
2.2 HydroLAKES Ecosystem Inventory	11
2.3 Sentinel-2	12
2.4 Temporal Coverage	13
2.5 Satellite Bands	13
2.6 Dynamic World	13
2.7 Environmental Indices	13
2.7.1 NDVI	13
2.7.2 NDWI	14
2.7.3 NBR	14
2.8 Data Quality Requirements	14
3 Detailed EcoTwin Methodology	15
3.1 End-to-End Pipeline	15
3.2 Step 1: Lake Boundary Retrieval	15
3.3 Step 2: Buffering	15

3.4	Step 3: Projected Grid Generation	16
3.5	Step 4: Earth Engine Collection Selection	16
3.6	Step 5: Cloud and Invalid-Pixel Handling	16
3.7	Step 6: Annual Composite Construction	17
3.8	Step 7: GeoTIFF Export	17
3.9	Step 8: Tile Completion and Resumability	17
3.10	Step 9: Local Raster Merging	17
3.11	Step 10: Analysis-Cell Generation	18
3.12	Step 11: Model Input Preparation	18
3.13	Step 12: Prithvi Embedding Generation	18
3.14	Step 13: Embedding Validation	19
3.15	Step 14: Vector Persistence	19
3.16	Step 15: Similarity Search	19
3.17	Cell-to-Lake Interpretation	20
3.18	Step 16: Temporal Profile Generation	20
3.19	Step 17: Analog-Based Forecasting Concept	20
3.20	Why the Analog Architecture is Explainable	21
4	System Architecture and Implementation	22
4.1	Architecture Overview	22
4.2	Technology Stack	23
4.3	Offline and Online Separation	23
4.3.1	Offline Path	23
4.3.2	Online Path	24
4.4	PostgreSQL and PostGIS	24
4.5	pgvector	24
4.6	Core Database Entities	24
4.6.1	Lakes	25
4.6.2	Lake Images	25
4.6.3	Lake Tiles	25
4.6.4	Sub-Regions	25
4.6.5	Sub-Region Features	25
4.7	Backend Services	25
4.8	Frontend	26
4.9	Map Rendering and Geometry Provenance	26

4.10	Asynchronous Processing	26
4.11	Dockerized Development Environment	27
4.12	Configuration Management	27
5	Experimental Results and Engineering Evaluation	28
5.1	Current Processing Cohort	28
5.2	Temporal Availability Observations	29
5.3	Embedding Pipeline Performance	29
5.4	Similarity Search Demonstration	29
5.5	Interpretation of Similarity Scores	30
5.6	Temporal Profile Results	30
5.7	Processing Reliability	31
5.8	Engineering Problems Encountered	31
5.8.1	Earth Engine Memory Limits	31
5.8.2	Earth Engine Concurrency Limits	31
5.8.3	Historical Band Differences	31
5.8.4	Large Raster Storage	32
5.8.5	Disk Constraints	32
5.8.6	Network and Authentication Constraints	32
5.9	Data Migration Between Team Members	32
6	Discussion	33
6.1	Why Foundation-Model Embeddings	33
6.2	Why Spatial Cells Instead of One Lake Image	33
6.3	Why Local Raster Processing	33
6.4	Why Vector Database Search	34
6.5	Spatial Similarity versus Temporal Similarity	34
6.6	Strengths	34
6.7	Limitations	35
6.7.1	Observation Availability	35
6.7.2	Optical Cloud Dependence	35
6.7.3	Embedding Semantics	35
6.7.4	Spatial Grid Dependence	35
6.7.5	Model Input Dependence	35
6.7.6	Forecasting Dependence on Analogs	35

6.8	Validation Strategy	35
7	Analog-Based Forecasting	37
7.1	Forecasting Philosophy	37
7.2	Historical Twin Alignment	37
7.3	Multiple Analog Forecast	37
7.4	Forecasted Variables	38
7.5	Forecast Uncertainty	38
7.6	Forecast Evaluation	38
8	Conclusion and Future Work	40
8.1	Conclusion	40
8.2	Future Work	40
8.3	Final Remarks	41
A	Database Schema	43
A.1	Lake Table	43
A.2	Lake Images	44
A.3	Lake Tiles	44
A.4	Sub-Regions	44
A.5	Embeddings	45
B	Data Processing State Machine	46
C	Representative API Structure	47
D	Database Migration Procedure	48
D.1	Backup	48
D.2	Temporary Restore	48
D.3	ETL Validation	49
D.4	Production Merge	49
D.5	Cleanup	49
E	Recommended Final Report Figures	50
F	Final Experimental Tables	51

List of Figures

4.1	Recommended figure: EcoTwin end-to-end architecture showing HydroLAKES, Google Earth Engine, local raster processing, Prithvi, PostgreSQL/PostGIS/pgvector, FastAPI, and React/Leaflet.	22
-----	---	----

List of Tables

2.1	HydroLAKES-derived information used by EcoTwin.	12
3.1	Sentinel-2 collection strategy.	16
4.1	EcoTwin implementation stack.	23
5.1	Current EcoTwin processing configuration.	28
5.2	Representative similarity-search demonstration.	30
A.1	Principal lake table fields.	43
A.2	Lake imagery metadata.	44
A.3	Intermediate imagery-tile metadata.	44
A.4	Analysis-cell representation.	44
C.1	Representative EcoTwin API groups.	47
F.1	Final production statistics to be reported.	51
F.2	Final performance metrics to be reported.	52

Chapter 1

Introduction

1.1 Background

Earth observation satellites provide repeated measurements of ecosystems at spatial scales that cannot be obtained economically through field observations alone. Sentinel-2 is particularly useful for this purpose because it provides multispectral optical observations suitable for vegetation, water, land-cover, and surface-condition analysis.

A conventional ecosystem-analysis workflow normally begins with a specific question such as vegetation change, water extent, or land cover. EcoTwin approaches the problem from a complementary direction: instead of analysing a lake only in isolation, it first asks whether another ecosystem in a large reference collection looks similar in its spatial-spectral representation.

The project therefore treats ecosystem comparison as a retrieval problem. Satellite observations are converted into learned feature vectors, and those vectors are indexed so that an ecosystem can be compared against a large collection without repeatedly processing the raw imagery.

1.2 Problem Statement

The practical problem is to identify geographically distributed ecosystems that are similar to a selected target ecosystem using information contained in satellite observations.

The problem has several engineering constraints:

1. lake boundaries are irregular polygons rather than fixed-size image squares;
2. large satellite scenes are expensive to download and store;
3. historical Sentinel-2 collections do not expose identical quality bands across all years;
4. cloud and invalid-pixel contamination must be controlled;

5. large ecosystems may require multiple image tiles;
6. the embedding pipeline must process many spatial regions;
7. embeddings must remain associated with their source geometries and lake/year metadata; and
8. online similarity search must be substantially cheaper than recomputing embeddings.

1.3 Core EcoTwin Hypothesis

EcoTwin is based on the operational hypothesis:

Ecosystems elsewhere that are structurally similar in geospatial feature space can provide useful analogs for understanding the spatial and historical behaviour of a target ecosystem.

The hypothesis separates two tasks:

1. **Analog discovery:** find ecosystems that resemble the target in the learned spatial-spectral representation.
2. **Trajectory analysis:** compare historical environmental indicators of the target and its analogs.

1.4 Research Objectives

The project objectives are:

1. construct a scalable geospatial data-ingestion pipeline;
2. represent lake environments using standardized satellite image regions;
3. generate fixed-dimensional Prithvi embeddings;
4. store and index embeddings using PostgreSQL and pgvector;
5. retrieve ranked analog ecosystems using cosine similarity;
6. expose lake boundaries and analogs through an interactive geospatial interface;
7. compare historical NDVI, NDWI, and NBR profiles; and
8. extend analog retrieval into an empirical forecasting mechanism based on historical twin trajectories.

1.5 Project Evolution During Development

The implementation evolved considerably from the initial conceptual design. The early design considered standardized ecosystem blocks of approximately 5×5 km. During implementation, the data engineering problem became more important than the initial geometric assumption. Large lake polygons and Earth Engine export limits made it preferable to separate *imagery acquisition tiles* from *analysis cells*.

The current implementation therefore uses a lake-centred buffered imagery extraction stage followed by local raster merging and approximately 1×1 km analysis cells. The larger imagery tiles are an I/O and Earth Engine management mechanism; the smaller cells are the units from which embeddings are generated. This distinction allows large lakes to be processed without forcing the foundation model to operate directly on irregular polygons.

1.6 Contributions

The project contributions are:

- an end-to-end lake-to-embedding-to-analog pipeline;
- a robust satellite-image acquisition strategy using Google Earth Engine and local raster processing;
- polygon-aware spatial subdivision and coverage filtering;
- Prithvi-based six-band geospatial representations;
- persistent 768-dimensional vector storage;
- cosine-similarity analog retrieval;
- temporal indicator comparison;
- a FastAPI backend and React/Leaflet frontend; and
- an engineering workflow for validation and database consolidation across multiple development environments.

1.7 Report Organization

Chapter 2 describes the source datasets and data engineering process. Chapter 3 presents the complete scientific pipeline. Chapter 4 describes the software and database implementation. Chapter 5 documents the current experimental outcomes and engineering observations. Chapter 6 discusses interpretation and limitations. Chapter 7 describes the analog-based forecasting layer. Chapter 8 summarizes the project and future work.

Chapter 2

Data Sources and Data Engineering

2.1 Overview

EcoTwin combines static geographic information with repeated satellite observations. The static layer defines *where* an ecosystem is, while the satellite layer defines *what the ecosystem looks like through time*.

The data-engineering pipeline was designed so that expensive satellite processing is performed offline. The online application then operates primarily on database records, precomputed features, and indexed embeddings.

2.2 HydroLAKES Ecosystem Inventory

HydroLAKES provides lake polygons and associated metadata. The database representation used by EcoTwin retains fields including:

- `hylak_id` / lake identifier;
- lake name;
- country;
- lake area;
- elevation;
- pour-point latitude and longitude;
- polygon geometry;
- lake type;
- average depth;

- total volume; and
- watershed area.

The complete HydroLAKES-derived inventory is much larger than the cohort used for the computationally expensive processing stages. The development pipeline was narrowed to a selected processing cohort of lakes so that multi-year imagery and embeddings could be generated within available storage, Earth Engine, and compute budgets.

Table 2.1: HydroLAKES-derived information used by EcoTwin.

Field	Type	Role in EcoTwin
Lake ID	Integer	Primary ecosystem identifier
Lake name	Text	Human-readable identification
Country	Text	Geographic filtering
Lake area	Numeric	Size and metadata analysis
Elevation	Numeric	Environmental context
Pour latitude/longitude	Numeric	Geographic reference
Geometry	PostGIS geometry	Spatial processing and mapping
Lake type	Categorical	Ecosystem metadata
Average depth	Numeric	Descriptive context
Total volume	Numeric	Descriptive context
Watershed area	Numeric	Catchment context

2.3 Sentinel-2

Sentinel-2 multispectral imagery is the primary remote-sensing input. Google Earth Engine is used for collection filtering, compositing, masking, and export. The principal collection used during development was:

COPERNICUS/S2_SR_HARMONIZED

For years or locations where the surface-reflectance harmonized collection did not provide observations, the pipeline introduced a fallback to:

COPERNICUS/S2

This fallback was particularly important for early Sentinel-2 years.

2.4 Temporal Coverage

The initial analytical design focused on a 2018–2025 operational window. During implementation, ingestion was extended to earlier Sentinel-2 observations where available, resulting in an attempted 2016–2025 extraction window.

The practical coverage is therefore lake- and year-dependent. Early years contain fewer observations and, in some cases, require the Level-1C fallback collection. The pipeline records the collection actually used so that missing or fallback observations are not silently treated as equivalent to the primary surface reflectance product.

2.5 Satellite Bands

The current embedding input uses six bands:

$$B2, B3, B4, B5, B6, B7$$

These correspond to the blue, green, red, narrow near-infrared, and two short-wave infrared channels used by the selected Prithvi configuration.

The bands are selected consistently from the exported GeoTIFFs before model inference.

2.6 Dynamic World

Dynamic World was evaluated as a complementary land-cover/context source. It can provide 10 m land-use/land-cover information and can be used to augment interpretation of the spatial analogs.

In the current EcoTwin architecture, Sentinel-2 is the principal source for the embedding image, while Dynamic World is treated as a contextual layer rather than as a replacement for the Prithvi input.

2.7 Environmental Indices

EcoTwin maintains temporal environmental indicators for ecosystem comparison.

2.7.1 NDVI

$$NDVI = \frac{NIR - Red}{NIR + Red} \quad (2.1)$$

NDVI provides a vegetation-sensitive signal and is useful for studying seasonal and interannual vegetation behaviour.

2.7.2 NDWI

The water-sensitive index is computed as:

$$NDWI = \frac{Green - NIR}{Green + NIR} \quad (2.2)$$

It provides a complementary signal for surface-water behaviour.

2.7.3 NBR

The Normalized Burn Ratio is:

$$NBR = \frac{NIR - SWIR}{NIR + SWIR} \quad (2.3)$$

NBR provides information related to surface disturbance and burn-related spectral change.

The exact band mapping used for each index is kept consistent with the final feature-extraction implementation.

2.8 Data Quality Requirements

A valid ecosystem record requires:

1. a valid lake polygon;
2. a valid year;
3. a successful imagery query;
4. a usable composite;
5. valid exported raster data;
6. valid spatial cells after coverage filtering; and
7. a successfully generated embedding for each retained cell.

The pipeline explicitly distinguishes successful, failed, skipped, and already-completed work so that long-running jobs can be resumed.

Chapter 3

Detailed EcoTwin Methodology

3.1 End-to-End Pipeline

The implemented workflow is:

$$\begin{aligned} & \textit{Lake Inventory} \rightarrow \textit{Boundary Validation} \\ & \quad \rightarrow \textit{Buffering} \rightarrow \textit{Imagery Tiling} \\ & \quad \rightarrow \textit{Sentinel-2 Composite} \rightarrow \textit{GeoTIFF} \\ & \quad \rightarrow \textit{Local Mosaic} \rightarrow \textit{1 km Cells} \\ & \quad \rightarrow \textit{Coverage Filter} \rightarrow \textit{Prithvi} \\ & \quad \rightarrow \textit{768D Embeddings} \rightarrow \textit{pgvector} \\ & \quad \rightarrow \textit{Cosine Search} \rightarrow \textit{Analog} \\ & \quad \rightarrow \textit{Temporal Profiles} \rightarrow \textit{Forecast} \end{aligned} \tag{3.1}$$

3.2 Step 1: Lake Boundary Retrieval

The pipeline begins by reading the target lake polygon from PostGIS. Keeping the original geometry in a spatial database avoids repeated conversion between shapefiles and application-specific formats.

The geometry is checked for validity and transformed into an appropriate projected CRS before distance-based operations are performed.

3.3 Step 2: Buffering

The lake polygon is expanded using a configurable buffer. During development, a 1200 m buffer was used for the imagery extraction stage.

The buffer is important because the ecosystem representation is not restricted to the water polygon itself. The surrounding land, shoreline, vegetation, and local landscape context contribute to the spatial-spectral representation.

3.4 Step 3: Projected Grid Generation

The buffered polygon is projected into a metric CRS. A regular imagery grid is then generated around the buffered geometry.

The imagery grid is deliberately different from the final analysis grid. During the latest extraction implementation, an approximately 8500 m tile size was used to balance Earth Engine request size, memory use, and local processing cost.

A lake may therefore produce one, two, or several imagery tiles depending on its size and shape.

3.5 Step 4: Earth Engine Collection Selection

For each lake-year combination, the pipeline first queries the preferred surface-reflectance collection. If no usable observation exists, the system falls back to the Level-1C collection.

This decision is made before the export request and is recorded in the processing metadata.

Table 3.1: Sentinel-2 collection strategy.

Priority	Collection	Purpose
1	COPERNICUS/S2_SR_HARMONIZED	Preferred surface reflectance
2	COPERNICUS/S2	Fallback for early or unavailable surface-reflectance years

3.6 Step 5: Cloud and Invalid-Pixel Handling

Cloud contamination is one of the main sources of noise in optical remote sensing. The current preprocessing pipeline uses cloud-probability information and class-based masking where the required bands are available.

The development configuration used a cloud-probability threshold of approximately 40%. Pixels classified as unusable are masked before compositing.

The implementation had to account for historical collection differences. Early Sentinel-2 products exposed different quality bands; code that assumed the presence of `QA60`, `MSK_CLASS1_OPAQUE`, or `MSK_CLASS1_CIRRUS` could fail when the selected collection did not expose the expected band. The final pipeline therefore treats cloud masking as a collection-aware operation rather than assuming one universal quality-band schema.

3.7 Step 6: Annual Composite Construction

For each lake-year-region, the filtered observations are combined into an annual composite.

The median operation is used to reduce the influence of residual clouds, outliers, and individual acquisition anomalies:

$$I_{annual}(p, b) = median(I_1(p, b), I_2(p, b), \dots, I_n(p, b)) \quad (3.2)$$

where p is a pixel and b is a selected spectral band.

The composite is clipped to the requested geographic extent before export.

3.8 Step 7: GeoTIFF Export

The selected six bands are exported as local GeoTIFF data. The development pipeline used 10 m spatial output for the selected Sentinel-2 bands and stored the raster in floating-point form.

Local storage has two major advantages:

1. the same Earth Engine composite does not need to be recomputed for every downstream analysis cell; and
2. embedding generation can be repeated without repeatedly consuming Earth Engine export capacity.

3.9 Step 8: Tile Completion and Resumability

Each imagery tile is represented by metadata including lake ID, year, tile index, geometry/bounding box, status, and local image path.

The pipeline checks whether a tile has already completed before starting a new Earth Engine request. This makes the large batch pipeline resumable after failures or interruptions.

3.10 Step 9: Local Raster Merging

After all tiles belonging to a lake-year are available, the local processing stage merges them into a single raster mosaic.

Rasterio is used for reading, reprojection/metadata handling, mosaicking, and writing. Compression settings are applied to reduce storage requirements while preserving the numerical data needed for model inference.

The merged image becomes the stable source from which smaller analysis cells are extracted.

3.11 Step 10: Analysis-Cell Generation

The merged lake image is intersected with a regular approximately 1×1 km analysis grid.

For each cell, the lake coverage is computed:

$$Coverage(c) = \frac{Area(c \cap Lake)}{Area(c)} \times 100 \quad (3.3)$$

Only cells above the configured minimum coverage threshold are retained. The development configuration used a threshold of approximately 15%.

This prevents cells that only touch a small corner of a lake from being treated as representative ecosystem regions.

3.12 Step 11: Model Input Preparation

Each retained analysis cell is read from the local merged GeoTIFF. The six selected bands are converted into the model’s expected tensor layout, numerical range, and spatial dimensions.

The processing stage therefore separates:

1. geographic geometry;
2. raster extraction;
3. model preprocessing; and
4. model inference.

This separation makes it possible to change the model without re-downloading the underlying satellite data.

3.13 Step 12: Prithvi Embedding Generation

Prithvi is used as the geospatial foundation model. Each retained analysis cell is transformed into a fixed-length feature vector.

For an input image X :

$$f(X) \rightarrow \mathbf{z}, \quad \mathbf{z} \in \mathbb{R}^{768} \quad (3.4)$$

The six-band configuration used by EcoTwin is based on B2-B7, corresponding to the selected blue, green, red, narrow-NIR, SWIR1, and SWIR2 channels.

The inference pipeline supports batched processing. During the development runs, a batch size of eight was used in CPU-based inference.

3.14 Step 13: Embedding Validation

Before insertion into the database, every embedding is checked for:

- expected dimensionality;
- missing or null values;
- numerical validity;
- correct lake association;
- correct year association; and
- correct spatial-cell association.

A failed inference is not inserted as a valid vector.

3.15 Step 14: Vector Persistence

The final embedding is stored in PostgreSQL using pgvector.

The conceptual relationship is:

$$Lake \rightarrow Year \rightarrow Spatial\ Cell \rightarrow Embedding \quad (3.5)$$

This preserves the provenance required to explain why an ecosystem was retrieved as an analog.

3.16 Step 15: Similarity Search

For a query embedding $\mathbf{q}$ and candidate embedding $\mathbf{x}$, EcoTwin uses cosine similarity:

$$sim(\mathbf{q}, \mathbf{x}) = \frac{\mathbf{q}^T \mathbf{x}}{\|\mathbf{q}\|_2 \|\mathbf{x}\|_2} \quad (3.6)$$

The candidates are ranked by decreasing similarity:

$$s_{(1)} \geq s_{(2)} \geq \dots \geq s_{(K)} \quad (3.7)$$

The highest-ranked ecosystems become the analog set displayed by the application.

3.17 Cell-to-Lake Interpretation

Because a lake can contain multiple analysis cells, the retrieval layer must preserve the relationship between cell-level embeddings and lake-level identity.

This design has two advantages:

1. it preserves local spatial detail instead of collapsing a large lake into one opaque vector too early; and
2. it allows retrieved similarities to be traced back to the actual spatial region that produced the embedding.

Where a single lake-level representation is required, cell representations can be aggregated using a coverage- or area-aware strategy. The cell-level vectors remain the underlying evidence.

3.18 Step 16: Temporal Profile Generation

For the query lake and retrieved analogs, historical environmental records are assembled into time series.

The primary indicators are:

- NDVI;
- NDWI; and
- NBR.

The interface can show the indicators independently or as a multi-index comparison.

3.19 Step 17: Analog-Based Forecasting Concept

EcoTwin’s forecasting design is not based on generating synthetic future satellite images. Instead, it uses the observed historical trajectory of an analogous ecosystem as empirical evidence.

Let the target indicator be y_t and an analog indicator be a_t . A simple trajectory-offset formulation is:

$$\Delta_t = y_t - a_t \tag{3.8}$$

The future analog trajectory can then be transferred to the target through an estimated historical offset:

$$\hat{y}_{t+h} = a_{t+h} + \hat{\Delta} \tag{3.9}$$

For multiple analogs, a weighted combination can be used:

$$\hat{y}_{t+h} = \sum_{i=1}^K w_i (a_{i,t+h} + \Delta_i) \quad (3.10)$$

with weights related to the similarity scores and normalized so that

$$\sum_{i=1}^K w_i = 1. \quad (3.11)$$

This formulation keeps the forecast grounded in historically observed ecosystem trajectories.

3.20 Why the Analog Architecture is Explainable

A parametric time-series model produces a prediction through learned parameters that can be difficult to connect to a particular ecosystem.

EcoTwin instead provides an explicit chain:

$$\text{Target} \rightarrow \text{Retrieved Twin} \rightarrow \text{Similarity} \rightarrow \text{Historical Trajectory} \rightarrow \text{Projected Trajectory} \quad (3.12)$$

This makes it possible to inspect the analogs responsible for a forecast and compare their historical behaviour with the target.

Chapter 4

System Architecture and Implementation

4.1 Architecture Overview

EcoTwin is implemented as a multi-component system. The offline pipeline performs the computationally expensive work, while the online application retrieves precomputed data.

Figure placeholder

Recommended figure: EcoTwin end-to-end architecture showing HydroLAKES, Google Earth Engine, local raster processing, Prithvi, PostgreSQL/PostGIS/pgvector, FastAPI, and React/Leaflet.

Figure 4.1: Recommended figure: EcoTwin end-to-end architecture showing HydroLAKES, Google Earth Engine, local raster processing, Prithvi, PostgreSQL/PostGIS/pgvector, FastAPI, and React/Leaflet.

4.2 Technology Stack

Table 4.1: EcoTwin implementation stack.

Layer	Technology	Responsibility
Frontend	React, Vite, TypeScript	Interactive application
Mapping	Leaflet / React-Leaflet	Spatial visualization
Backend	FastAPI, Python	REST services and orchestration
Database	PostgreSQL	Relational persistence
Spatial database	PostGIS	Polygon and spatial queries
Vector search	pgvector	Embedding storage and similarity search
Remote sensing	Google Earth Engine	Satellite filtering/compositing
Raster processing	Rasterio / Python	Local GeoTIFF processing
ML inference	Prithvi	Geospatial embeddings
Task queue	Celery / Redis	Asynchronous processing
Monitoring	Flower	Celery task monitoring
Administration	pgAdmin	Database administration
Containerization	Docker Compose	Reproducible local deployment

4.3 Offline and Online Separation

The system is divided into two logical paths.

4.3.1 Offline Path

The offline path performs:

1. lake preparation;
2. imagery acquisition;
3. raster merging;
4. spatial-cell extraction;
5. embedding generation;
6. temporal-feature preparation; and
7. vector indexing.

4.3.2 Online Path

The online path performs:

1. lake lookup;
2. map rendering;
3. similarity query;
4. analog profile retrieval;
5. temporal-series retrieval; and
6. report generation.

This architecture avoids running Earth Engine or the foundation model during every user request.

4.4 PostgreSQL and PostGIS

PostgreSQL is the central persistent store. PostGIS stores the original lake geometry, analysis-cell geometry, and spatial metadata.

Spatial information remains in the database rather than being converted into plain latitude/longitude tables. This allows intersection, containment, area, and map-query operations to be performed directly in the database.

4.5 pgvector

The pgvector extension stores the 768-dimensional Prithvi vectors.

The vector table maintains the association between:

$$Embedding \leftrightarrow Subregion \leftrightarrow Lake \leftrightarrow Year \tag{4.1}$$

Cosine-distance/similarity operations can therefore be executed directly against the indexed database rather than exporting vectors to CSV or recomputing features.

4.6 Core Database Entities

The implementation evolved around entities including:

- `lakes`;
- `lake_tiles`;

- `lake_images`;
- `sub_regions`;
- `sub_region_features`; and
- embedding/vector records.

4.6.1 Lakes

Stores the master ecosystem identity, metadata, and PostGIS geometry.

4.6.2 Lake Images

Stores the merged or processed imagery associated with a lake-year, including local path and processing status.

4.6.3 Lake Tiles

Stores the intermediate imagery tiles generated for a lake-year, including tile index, geometry, bounding box, status, and image path.

4.6.4 Sub-Regions

Stores the smaller analysis cells generated from the merged lake image, including geometry and coverage information.

4.6.5 Sub-Region Features

Stores model-derived features and embeddings associated with individual spatial cells.

4.7 Backend Services

The FastAPI backend provides services for:

- lake metadata lookup;
- nearest lake/grid-cell lookup;
- analog similarity search;
- temporal feature retrieval;
- forecast retrieval;

- report generation; and
- health/system status.

The API separates data retrieval from heavy batch processing. Long operations are handled asynchronously where appropriate.

4.8 Frontend

The frontend is implemented using React, Vite, and TypeScript. The map uses Leaflet/React-Leaflet.

The main user interaction is:

1. select or search for a lake;
2. display its geographic boundary;
3. inspect lake metadata;
4. request analogous ecosystems;
5. visualize the analog locations;
6. inspect similarity scores;
7. inspect analog profiles; and
8. compare temporal indicators and forecast information.

4.9 Map Rendering and Geometry Provenance

An important implementation requirement is that displayed lake boundaries must come from the same PostGIS geometry used for the analysis. Analog markers are derived from their stored geographic coordinates and lake identities.

This prevents a common failure mode in which the analytical result and the displayed map use different coordinate transformations or different numbering systems.

4.10 Asynchronous Processing

Celery and Redis are used for background processing. This is important because imagery downloads, raster merging, and model inference can take substantially longer than a normal HTTP request.

Flower is used to inspect worker activity and task progress during development.

4.11 Dockerized Development Environment

The local development environment was organized as a multi-service Docker Compose deployment containing services such as:

- PostgreSQL;
- Redis;
- FastAPI backend;
- Celery worker;
- pipeline worker;
- Flower; and
- pgAdmin.

The containerized design makes service dependencies explicit and allows the same architecture to be reproduced on another machine.

4.12 Configuration Management

The processing pipeline uses environment/configuration values for items such as:

- Earth Engine project/authentication;
- date ranges;
- cloud thresholds;
- imagery buffer distance;
- imagery tile size;
- analysis-cell size;
- coverage threshold;
- local storage paths;
- database credentials; and
- worker/batch settings.

This prevents experiment-specific values from being hard-coded throughout the processing code.

Chapter 5

Experimental Results and Engineering Evaluation

5.1 Current Processing Cohort

During development, a processing cohort of approximately 829 lakes was targeted for multi-year processing. The wider HydroLAKES-derived Indian inventory is substantially larger, so the 829-lake cohort represents the computationally processed subset rather than the entire source inventory.

Table 5.1: Current EcoTwin processing configuration.

Parameter	Current configuration
Lake source	HydroLAKES
Target processing cohort	~829 lakes
Imagery window	2016–2025 attempted
Primary Sentinel collection	S2 SR Harmonized
Fallback collection	S2 Level-1C
Imagery buffer	1200 m
Imagery tile size	approximately 8500 m
Analysis cell	approximately 1 km $\times$ 1 km
Coverage threshold	approximately 15%
Embedding model	Prithvi
Embedding dimension	768
Input bands	B2–B7
Similarity metric	Cosine similarity

5.2 Temporal Availability Observations

The Earth Engine collection checks demonstrated that early years have substantially less usable surface-reflectance coverage than recent years. In particular, the 2016–2017 period required special handling, while later years provided much denser observation sets.

This is an important distinction between the *requested* temporal window and the *available valid observations*. EcoTwin therefore stores year-level processing status rather than assuming that every lake has a complete ten-year sequence.

5.3 Embedding Pipeline Performance

The embedding stage was implemented as batch inference. During development, inference logs showed CPU execution with a batch size of eight.

The pipeline is designed so that:

$$N_{\text{embeddings}} = N_{\text{valid cells}} \tag{5.1}$$

rather than

$$N_{\text{embeddings}} = N_{\text{lakes}}. \tag{5.2}$$

This distinction is fundamental to EcoTwin: a large lake can generate several spatial embeddings, and the vector index therefore contains spatially localized ecosystem representations.

5.4 Similarity Search Demonstration

A representative similarity-analysis run used a large candidate collection and returned a ranked set of analogs. In a previously observed application run, approximately 12,458 candidate lake/region records were compared, the interface returned the top 10 analogs, and the best displayed similarity was approximately 0.942.

These values should be treated as *current demonstration results* rather than final benchmark claims until the final production database is frozen and the experiment is repeated under a documented configuration.

Table 5.2: Representative similarity-search demonstration.

Metric	Observed value
Candidate records	~12,458
Returned analogs	10
Best displayed similarity	~0.942
Similarity metric	Cosine similarity

5.5 Interpretation of Similarity Scores

A similarity score indicates closeness in the learned representation space. It does not directly imply identical ecological processes.

A high score can therefore be interpreted as:

The target and analog contain similar spatial-spectral information under the selected satellite bands, preprocessing procedure, and Prithvi representation.

Ecological interpretation requires additional evidence from lake metadata, spatial maps, and temporal environmental indicators.

5.6 Temporal Profile Results

The temporal layer provides a second axis of validation. Two lakes can be spatially similar but exhibit different historical trajectories, while two lakes with different geographic locations may show similar seasonal behaviour.

EcoTwin therefore presents:

- NDVI trajectories;
- NDWI trajectories;
- NBR trajectories;
- observation availability;
- yearly summaries; and
- comparison against retrieved analogs.

5.7 Processing Reliability

The system was intentionally designed to tolerate partial failure.

A lake-year can be in states such as:

- pending;
- processing;
- completed;
- failed; or
- skipped because output already exists.

This makes large multi-year extraction jobs restartable instead of requiring a complete restart after a single failed Earth Engine request.

5.8 Engineering Problems Encountered

5.8.1 Earth Engine Memory Limits

Large requests sometimes produced Earth Engine memory-limit errors. The solution was to reduce the size of each export region and to move part of the processing to local raster operations.

5.8.2 Earth Engine Concurrency Limits

Parallel workers caused HTTP 429 errors when too many requests were issued simultaneously. Connection-pool saturation was also observed.

The pipeline was consequently changed toward smaller controlled worker counts, resumable jobs, and local reuse of already-generated imagery.

5.8.3 Historical Band Differences

The early Sentinel-2 Level-1C fallback exposed a different set of quality bands from assumptions made for other collections. For example, requests that selected `MSK_CLASSI_CIRRUS`, `MSK_CLASSI_OPAQUE`, or `QA60` could fail depending on the actual image schema.

This led to collection-aware preprocessing rather than a single hard-coded cloud-mask implementation.

5.8.4 Large Raster Storage

Merged rasters can become much larger than individual source tiles. Compression, floating-point type control, tiled GeoTIFF writing, and local reuse were therefore important.

5.8.5 Disk Constraints

During development, the local root filesystem reached 100% utilization, causing raster and Docker operations to fail with “No space left on device”. This demonstrated that the data pipeline must account for intermediate storage, not only final dataset size.

5.8.6 Network and Authentication Constraints

Earth Engine authentication and access depend on external network services. Authentication failures and inability to reach OAuth endpoints temporarily prevented pipeline execution.

The implementation therefore separates authentication from per-lake processing and records failures instead of silently producing incomplete datasets.

5.9 Data Migration Between Team Members

Because the project was developed by multiple team members, a database consolidation procedure was designed.

Each developer can create a PostgreSQL custom-format dump. The dump is restored into a temporary database, validated through a Python ETL process, and then merged into the production database.

The procedure preserves:

- PostGIS geometries;
- pgvector embeddings;
- indexes;
- constraints; and
- PostgreSQL data types.

This approach is safer than CSV-based migration because CSV does not naturally preserve PostGIS geometry and vector types.

Chapter 6

Discussion

6.1 Why Foundation-Model Embeddings

Traditional similarity systems require manually specifying which variables should define similarity. EcoTwin instead uses a learned geospatial representation derived from multispectral imagery.

This changes the problem from:

$$\textit{Similarity} = f(\text{manually selected attributes})$$

to:

$$\textit{Similarity} = f(\text{learned spatial-spectral representation})$$

The advantage is that spatial context can be represented jointly rather than through a small set of manually selected scalar attributes.

6.2 Why Spatial Cells Instead of One Lake Image

A lake is not spatially homogeneous. Shoreline vegetation, surrounding land cover, built-up regions, agricultural land, mountain slopes, and water surfaces can coexist in the same analysis area.

Using smaller cells preserves this heterogeneity.

The cell-level representation also improves explainability because a retrieved similarity can be associated with a geographic region rather than only with an entire lake.

6.3 Why Local Raster Processing

Downloading the complete India-scale satellite archive would be impractical for the project. EcoTwin therefore uses an “extract once, reuse many times” strategy:

Earth Engine → Local GeoTIFF → Multiple Downstream Features

This substantially reduces repeated remote requests.

6.4 Why Vector Database Search

Without vector persistence, every query would require model inference on the candidate ecosystem collection.

With pgvector:

Query Image → One Embedding → Vector Search → Top-K

The expensive foundation-model inference is therefore moved into the offline indexing stage.

6.5 Spatial Similarity versus Temporal Similarity

EcoTwin deliberately keeps spatial and temporal similarity conceptually separate.

Spatial similarity answers:

Which ecosystems look similar to the target in the learned representation?

Temporal similarity answers:

Do those ecosystems exhibit similar historical behaviour?

A robust analog should ideally provide useful evidence on both axes.

6.6 Strengths

The implemented system provides:

- scalable offline preprocessing;
- resumable Earth Engine extraction;
- explicit spatial provenance;
- foundation-model representations;
- persistent vector search;
- temporal contextualization;
- interactive geospatial visualization; and
- a path toward explainable analog-based forecasting.

6.7 Limitations

6.7.1 Observation Availability

The ten-year target window does not guarantee ten complete years for every lake. Historical collection availability and cloud filtering can create gaps.

6.7.2 Optical Cloud Dependence

Sentinel-2 is an optical sensor. Cloud cover and atmospheric conditions can reduce valid observations.

6.7.3 Embedding Semantics

A high embedding similarity does not guarantee ecological equivalence. The embedding is a representation of the selected satellite inputs and model.

6.7.4 Spatial Grid Dependence

Changing the analysis-cell size or coverage threshold changes the resulting embeddings. The spatial discretization is therefore a model parameter.

6.7.5 Model Input Dependence

Changing bands, preprocessing, normalization, temporal compositing, or image extent can change the embedding space.

6.7.6 Forecasting Dependence on Analogs

Analog forecasting is only as strong as the retrieved historical twins. Poor analogs lead to poor forecasts, even if the mathematical forecasting procedure itself is correct.

6.8 Validation Strategy

The system should be validated at four levels:

1. **Data validation:** check image completeness, geometry validity, and temporal availability.
2. **Representation validation:** verify embedding dimensionality and numerical validity.

3. **Retrieval validation:** inspect similarity rankings and compare retrieved maps and metadata.
4. **Temporal validation:** determine whether high-ranked analogs also provide useful historical trajectories.

Independent ecological or field observations would provide the strongest external validation but were not available as a complete ground-truth source during the present development cycle.

Chapter 7

Analog-Based Forecasting

7.1 Forecasting Philosophy

The EcoTwin forecasting layer follows the same analog architecture as the retrieval system. Instead of treating forecasting as an independent machine-learning problem, it reuses the evidence already identified by similarity search.

The logic is:

$$\text{Current Target} \rightarrow \text{Historical Twin} \rightarrow \text{Observed Future Trajectory} \rightarrow \text{Target Forecast} \quad (7.1)$$

7.2 Historical Twin Alignment

Suppose the target ecosystem has an indicator sequence Y_t and an analog has sequence A_t . A historical overlap period is used to estimate how the target differs from the analog.

A simple offset estimate is:

$$\hat{\Delta} = \frac{1}{T} \sum_{t=1}^T (Y_t - A_t) \quad (7.2)$$

The future analog trajectory is then adjusted using this offset.

7.3 Multiple Analog Forecast

For K analogs, each analog receives a weight w_i derived from its similarity. A generic normalization is:

$$w_i = \frac{s_i}{\sum_{j=1}^K s_j} \quad (7.3)$$

for non-negative similarity scores s_i .

The predicted trajectory becomes:

$$\hat{Y}_{t+h} = \sum_{i=1}^K w_i (A_{i,t+h} + \hat{\Delta}_i). \quad (7.4)$$

The application can therefore explain a prediction through the analog lakes that contributed to it.

7.4 Forecasted Variables

The principal targets are:

- NDVI;
- NDWI;
- NBR; and
- a combined ecosystem indicator derived from these series, where implemented.

7.5 Forecast Uncertainty

The spread among analog predictions can provide an empirical uncertainty signal.

For example, if the analog trajectories are widely separated, forecast confidence should be lower than when the trajectories closely agree.

This is not equivalent to a formally calibrated probabilistic forecast and should be described as an analog-dispersion measure unless a formal uncertainty model is added.

7.6 Forecast Evaluation

Once a sufficiently long historical sequence is available, a hindcasting experiment can be performed:

1. hide the latest observed period;
2. retrieve analogs using only information available before the cutoff;
3. generate a forecast;
4. compare the forecast against the held-out observations.

Recommended metrics include:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (7.5)$$

and

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (7.6)$$

The evaluation should be performed separately for NDVI, NDWI, and NBR rather than reporting only a single aggregate number.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

EcoTwin evolved from a conceptual ecosystem-comparison idea into an end-to-end geospatial retrieval system.

The final architecture separates the problem into several stages: a lake inventory defines the ecosystem boundary; Google Earth Engine provides multi-year Sentinel-2 observations; a buffered and tiled raster pipeline makes large exports manageable; local mosaicking creates reusable lake-year images; one-kilometre analysis cells provide spatially consistent units; Prithvi converts these cells into 768-dimensional representations; pgvector makes those representations searchable; and the web application exposes the retrieved analogs together with spatial and temporal context.

The most significant engineering contribution of the project is therefore not a single model component but the integration of geospatial data engineering, foundation-model inference, vector retrieval, temporal analysis, and interactive visualization into one reproducible workflow.

The development experience also demonstrated that large-scale Earth-observation systems require careful handling of external service limits, historical data differences, raster storage, concurrency, resumability, and database provenance.

8.2 Future Work

Future work should focus on:

1. completing and systematically evaluating the analog-based forecasting layer;
2. performing temporal hindcasting experiments;
3. calibrating forecast uncertainty using analog dispersion and historical error;
4. adding independent ecological validation;
5. improving the lake-level aggregation strategy;

6. experimenting with alternative cell sizes and coverage thresholds;
7. incorporating additional environmental variables;
8. improving temporal compositing;
9. supporting larger ecosystem inventories;
10. optimizing vector indexing for larger collections;
11. improving asynchronous pipeline orchestration; and
12. deploying a shared production database and hosted processing infrastructure.

8.3 Final Remarks

EcoTwin provides a practical bridge between modern geospatial foundation models and ecosystem-scale comparative analysis. Its central idea is to use historically observed ecosystems as computationally retrievable analogs rather than treating every ecosystem as an isolated forecasting problem.

Bibliography

- [1] N. Gorelick, M. Hancher, M. Dixon, S. Ilyushchenko, D. Thau, and R. Moore, “Google Earth Engine: Planetary-scale geospatial analysis for everyone,” *Remote Sensing of Environment*, vol. 202, pp. 18–27, 2017.
- [2] European Space Agency, “Sentinel-2 Mission,” Copernicus documentation.
- [3] HydroLAKES, Global lake database and associated lake-boundary information used as the static ecosystem inventory.
- [4] NASA and IBM, “Prithvi-EO: Geospatial foundation model for Earth observation,” official model documentation and associated publication.
- [5] C. F. Brown *et al.*, “Dynamic World, near real-time global 10 m land use land cover mapping,” 2022.
- [6] N. Pettorelli, J. O. Vik, A. Mysterud, J.-M. Gaillard, C. J. Tucker, and N. C. Stenseth, “Using the satellite-derived NDVI to assess ecological responses to environmental change,” *Trends in Ecology & Evolution*, vol. 20, no. 9, pp. 503–510, 2005.
- [7] A. Huete, K. Didan, T. Miura, E. P. Rodriguez, X. Gao, and L. G. Ferreira, “Overview of the radiometric and biophysical performance of the MODIS vegetation indices,” *Remote Sensing of Environment*, vol. 83, no. 1–2, pp. 195–213, 2002.

Appendix A

Database Schema

A.1 Lake Table

Table A.1: Principal lake table fields.

Column	Type	Description
lake_id	Integer	Unique lake identifier
lake_name	Text	Lake name
country	Text	Country
lake_area	Numeric	Lake area
elevation	Numeric	Elevation
pour_lat	Numeric	Pour-point latitude
pour_long	Numeric	Pour-point longitude
geometry	PostGIS Geometry	Lake boundary
lake_type	Text	Lake classification
depth_avg	Numeric	Average depth
vol_total	Numeric	Total volume
wshd_area	Numeric	Watershed area

A.2 Lake Images

Table A.2: Lake imagery metadata.

Field	Type	Description
lake_id	Integer	Source lake
year	Integer	Observation/composite year
image_path	Text	Local GeoTIFF path
status	Text	Processing status
collection_used	Text	Sentinel collection used

A.3 Lake Tiles

Table A.3: Intermediate imagery-tile metadata.

Field	Type	Description
tile_id	Integer	Tile identifier
lake_id	Integer	Source lake
year	Integer	Composite year
tile_index	Text	Grid position/index
tile_geometry	Geometry	Tile polygon
bbox	Geometry/Text	Export bounding box
status	Text	Tile state
image_path	Text	Local tile path
created_at	Timestamp	Creation time

A.4 Sub-Regions

Table A.4: Analysis-cell representation.

Field	Type	Description
sub_region_id	Integer	Unique cell identifier
lake_id	Integer	Parent lake
year	Integer	Associated imagery year
geometry	Geometry	Analysis-cell polygon
coverage_ratio	Numeric	Lake coverage within cell

A.5 Embeddings

Each valid analysis cell has an associated Prithvi embedding with 768 numerical dimensions. The vector is stored with sufficient metadata to recover its lake, year, and spatial region.

Appendix B

Data Processing State Machine

The processing state of an individual lake-year-tile follows the general pattern:

$$Pending \rightarrow Processing \rightarrow Completed$$

or:

$$Processing \rightarrow Failed$$

A previously completed item can be recognized and skipped:

$$Completed + Existing Output \rightarrow Skipped$$

This stateful design is important for large Earth Engine jobs because a failure in one lake-year should not invalidate successfully completed work.

Appendix C

Representative API Structure

Table C.1: Representative EcoTwin API groups.

Method	Endpoint family	Purpose
GET	Lake lookup	Retrieve lake metadata and geometry
GET	Similarity query	Retrieve analogous ecosystems
GET	Temporal profile	Retrieve NDVI/NDWI/NBR history
GET	Forecast	Retrieve analog-based forecast information
GET	Report	Retrieve report-generation information
GET	Health/status	Check service availability

Appendix D

Database Migration Procedure

The project requires safe consolidation of data generated by different team members.

D.1 Backup

Each developer creates a PostgreSQL custom-format dump:

```
1 pg_dump \  
2   -h localhost \  
3   -U postgres \  
4   -d ecotwin \  
5   -Fc \  
6   -f member.dump
```

The custom dump preserves PostgreSQL-native structures including PostGIS geometries and pgvector embeddings.

D.2 Temporary Restore

A temporary database is created:

```
1 createdb -h localhost -U postgres ecotwin_member
```

The dump is restored:

```
1 pg_restore \  
2   -h localhost \  
3   -U postgres \  
4   -d ecotwin_member \  
5   member.dump
```

D.3 ETL Validation

The ETL process reads from the temporary database and validates:

- missing embeddings;
- invalid geometries;
- duplicate records;
- vector dimensions;
- lake/year uniqueness;
- image-path existence; and
- expected row counts.

Validated records are then inserted into the production database.

D.4 Production Merge

Typical tables involved in consolidation include:

- `lake_embeddings`;
- `lake_tiles`; and
- `lake_images`.

Conflict handling can use database uniqueness constraints and `INSERT ... ON CONFLICT` logic where appropriate.

D.5 Cleanup

After validation, the temporary database can be removed:

```
1 dropdb -h localhost -U postgres ecotwin_member
```

Appendix E

Recommended Final Report Figures

The following figures should be inserted before final submission:

1. EcoTwin system architecture.
2. HydroLAKES study-area map.
3. Lake polygon and 1200 m buffer.
4. Approximately 8.5 km imagery tile grid.
5. Example Sentinel-2 annual composite.
6. Merged lake-year GeoTIFF.
7. 1 km analysis-cell grid and 15% coverage filtering.
8. Prithvi embedding pipeline.
9. PostgreSQL/PostGIS/pgvector schema.
10. Similarity-search workflow.
11. Query lake and Top- K analog map.
12. Analog lake profile cards.
13. NDVI temporal comparison.
14. NDWI temporal comparison.
15. NBR temporal comparison.
16. Analog-based forecast plot.
17. Frontend screenshots.
18. PDF/report-generation screenshot.

Appendix F

Final Experimental Tables

Before submission, populate the following values directly from the final production database and pipeline logs:

Table F.1: Final production statistics to be reported.

Metric	Final value
Source lakes	TBD
Processed lakes	TBD
Lake-years attempted	TBD
Lake-years completed	TBD
Imagery tiles generated	TBD
Merged lake images	TBD
Analysis cells generated	TBD
Valid analysis cells	TBD
Embeddings generated	TBD
Embedding failures	TBD
Indexed vectors	TBD
Temporal records	TBD

Table F.2: Final performance metrics to be reported.

Operation	Final measurement
Average imagery extraction time	TBD
Average raster merge time	TBD
Average embedding time/cell	TBD
Similarity query latency	TBD
Temporal retrieval latency	TBD
Report generation time	TBD
Total preprocessing time	TBD